CSE 4165 FINAL EXAM MASTER CHEAT SHEET • SIDE 1: ALL TYPES OF Q1 (JS) & Q2 (PHP)

Time: 1 hr 55 min | 30 Marks | Budget: Q1 30m • Q2 35m • Q3 40m • Zip & Submit 10m

A4 DUPLEX • SIDE 1

Q1 UNIVERSAL HTML SKELETON (NO CSS NEEDED)

HTML SKELETON

```
<!— index.html: 1 or 2 inputs + button + label placed below —>
<input type="text" id="inputA" placeholder="Name / Subject">
<input type="number" id="inputB" step="any" placeholder="Value / Hours / Score">
<button id="actionBtn" type="button">Add / Calculate</button>
<label id="outLabel"></label>
<script src="script.js"></script>
```

TYPE 1A: RUNNING TOTAL & GOAL TRACKER (Calorie, Study Hours)

TYPE 1A

```
const GOAL = 20; // Target goal (e.g. 2000 kcal, 20 hrs, 100 pts)
let total = 0, count = 0;
const historyList = []; // Declared OUTSIDE function so it persists!

function handleAction() {
  const name = document.getElementById("inputA").value.trim();
  const val = parseFloat(document.getElementById("inputB").value);
  if (!name || isNaN(val) || val ≤ 0) { alert("Enter valid input"); return; }

  count++; total += val;
  historyList.push(name); // To prevent duplicates: if (!historyList.includes(name)) historyList.push(name);
  const avg = total / count;

  // 4-Tier Feedback Ladder (Chained ≤ prevents decimal gaps)
  let msg = "";
  if (total ≤ 5)       msg = "Start practicing regularly!";
  else if (total ≤ 12) msg = "Good progress, keep it balanced!";
  else if (total ≤ 19) msg = "Almost at your limit!";
  else                 msg = "Goal reached successfully!";

  // Exam Signature Warning: > N entries without reaching goal
  if (count > 7 && total < GOAL) msg = "Increase study time / frequent snacking!";

  // Format array & output ONCE (prevents innerHTML overwrite bug!)
  const itemsFormatted = `[${historyList.join(", ")}]`;
  document.getElementById("outLabel").innerHTML =
    `Total: ${total} | Avg: ${parseFloat(avg.toFixed(2))} | Items: ${itemsFormatted} | ${msg}`;
  document.getElementById("inputA").value = "";
  document.getElementById("inputB").value = "";
}
document.getElementById("actionBtn").addEventListener("click", handleAction);
```

TYPE 1B: CRITERIA & POINTS SCORER (Password Strength)

TYPE 1B

```
let attempts = 0;
function checkScore() {
  const p = document.getElementById("inputA").value;
  attempts++;
  let score = 0;
  if (p.length ≥ 6) score += Math.floor((p.length - 6) / 2) * 10;
  if (/[A-Z]/.test(p))      score += 15; // Uppercase
  if (/[a-z]/.test(p))      score += 15; // Lowercase
  if (/[0-9]/.test(p))      score += 20; // Number
  if (/[!@#$%^&*]/.test(p)) score += 25; // Special char

  let lvl = (score ≤ 30) ? "Very Weak" : (score ≤ 50) ? "Weak" :
            (score ≤ 70) ? "Medium" : (score ≤ 90) ? "Strong" : "Very Strong";
  let out = (score ≥ 100) ? "Perfect Password! Success!" : `Score: ${score} - ${lvl}`;

  // Warning on > 8 attempts without reaching Strong (score < 71)
  if (attempts > 8 && score < 71) out += " | Need practice! Add uppercase, digits & symbols.";
  document.getElementById("outLabel").innerHTML = out;
}
document.getElementById("actionBtn").addEventListener("click", checkScore);
```

TYPE 1C: MULTI-INPUT SENSOR & FORMULA (Patient Vitals)

TYPE 1C

```
const listA = [], listB = [];
function computeFormula() {
  const vA = parseFloat(document.getElementById("inputA").value);
  const vB = parseFloat(document.getElementById("inputB").value);
  if (isNaN(vA) || isNaN(vB)) return;
  listA.push(vA); listB.push(vB);

  const avgA = listA.reduce((s, n) => s + n, 0) / listA.length;
  const avgB = listB.reduce((s, n) => s + n, 0) / listB.length;
  // Formula with absolute difference: Math.abs(vA - 80)
  const score = (100 - vB) * 2 + Math.abs(vA - 80) * 0.5;
  const status = (score ≤ 10) ? "SAFE" : (score ≤ 20) ? "WARNING" : "DANGER";

  document.getElementById("outLabel").innerHTML =
    `Avg A: ${avgA.toFixed(1)} | Avg B: ${avgB.toFixed(1)} | Score: ${score.toFixed(1)} | Status: ${status}`;
}
```

Q2 UNIVERSAL HTML FORM (EXACTLY 3 INPUTS)

FORM SKELETON

```
<!— form.html: form action MUST target calculate.php with method POST —>
<form action="calculate.php" method="POST">
  <label>Input 1: <input type="number" name="val1" step="any" required></label><br>
  <label>Input 2: <input type="number" name="val2" step="any" required></label><br>
  <label>Input 3: <input type="number" name="val3" step="any" required></label><br>
  <input type="submit" value="Calculate">
</form>
```

TYPE 2A: RESOURCE CEILING ALLOCATION & WASTE (Venues, Screens, Rover)

TYPE 2A

```
<?php // calculate.php - Type 2A
function calculateBooking($people, $costPerPerson, $capacity) {
  $unitsNeeded = ceil($people / $capacity); // Always round UP with ceil()
  $totalCapacity = $unitsNeeded * $capacity;
  $emptyUnits = $totalCapacity - $people;
  $wastedMoney = $emptyUnits * $costPerPerson;

  $unusedPct = ($totalCapacity > 0) ? ($emptyUnits / $totalCapacity) * 100 : 0;
  $status = ($unusedPct ≤ 10) ? "Efficient" : (($unusedPct ≤ 25) ? "Acceptable" : "Wasteful");

  return ['units' => $unitsNeeded, 'empty' => $emptyUnits, 'waste' => $wastedMoney, 'status' => $status];
}
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $v1 = floatval($_POST['val1']); $v2 = floatval($_POST['val2']); $v3 = floatval($_POST['val3']);
  $r = calculateBooking($v1, $v2, $v3);
  echo "<table border='1' cellpadding='6'>
        <tr><th>Demand</th><th>Rate</th><th>Cap</th><th>Units</th><th>Empty</th><th>Wasted BDT</th><th>Status</th></tr>
        <tr><td>$v1</td><td>$v2</td><td>$v3</td><td>{$r['units']}</td><td>{$r['empty']}</td>
            <td>" . number_format($r['waste'], 2) . "</td><td>{$r['status']}</td></tr></table>";
} ?>
```

TYPE 2B: PRODUCTION VS TARGET DIFFERENCE (Event Sales, Credits)

TYPE 2B

```
<?php // calculate.php - Type 2B
function evaluatePerformance($rate, $duration, $target) {
  $total = $rate * $duration; // items/day * days OR credits/course * courses
  $grade = ($total ≥ 500) ? "Excellent" : (($total ≥ 300) ? "Good" : (($total ≥ 150) ? "Average" : "Poor"));
  $diff = $total - $target;
  if ($diff == 0)     $msg = "Target met exactly (0 difference)";
  elseif ($diff > 0) $msg = "Above target by $diff";
  else               $msg = "Below target by " . abs($diff);
  return ['total' => $total, 'grade' => $grade, 'msg' => $msg];
} ?>
```

TYPE 2C: BEST 2 OF 3 CT MARKS & PASS/FAIL

TYPE 2C

```
<?php // calculate.php - Type 2C
function processExamMarks($ct1, $ct2, $ct3, $mid, $final) {
  // Best two CTs: subtract minimum from sum of three, divide by 2
  $bestTwoAvg = (($ct1 + $ct2 + $ct3) - min($ct1, $ct2, $ct3)) / 2.0;
  $totalMarks = $bestTwoAvg + $mid + $final;
  $status = ($totalMarks > 54) ? "Passed" : "Failed";
  return ['bestAvg' => $bestTwoAvg, 'total' => $totalMarks, 'status' => $status];
} ?>
```

FAST SYNTAX & GOTCHAS REFERENCE

JS Concat: `"A" + b` • **PHP Concat:** `$a . $b`	**PHP Variables:** Always start with `$`
JS Round: `n.toFixed(2)` (returns string!)	**PHP Round:** `ceil($x)`, `number_format($x, 2)`
JS Array: `a.push(x)`, `a.join(", ")`	**PHP Array:** `$a[] = x`, `implode(", ", $a)`
JS Regex: `/[A-Z]/.test(str)`	**PHP Abs:** `abs($a - $b)`, Min: `min($a, $b, $c)`

CSE 4165 FINAL EXAM MASTER CHEAT SHEET • SIDE 2: PHP & MYSQL (Q3) & COMPLETE 15 SQL ARCHETYPES

Total Marks: 10M (database.sql: 5M + solution.php: 5M) | All 15 SQL Archetypes Tagged (242-261) + Mapping Dictionary

A4 DUPLEX • SIDE 2

Q3 PART 1: database.sql SKELETON (UNIFIED ITEMS TABLE)

DATABASE.SQL

```
CREATE DATABASE IF NOT EXISTS exam_db;
USE exam_db;
DROP TABLE IF EXISTS items;
CREATE TABLE items (
  ItemID INT PRIMARY KEY,
  ItemName VARCHAR(100),
  Category VARCHAR(50),
  Price DECIMAL(10,2), Stock INT, Rating DECIMAL(3,1),
  SaleDate DATE, Status VARCHAR(50)
);
INSERT INTO items (ItemID, ItemName, Category, Price, Stock, Rating, Sal
eDate, Status) VALUES
(1, 'Laptop',     'Electronics', 50000.00, 5,   4.8, '2024-01-01', 'Sol
d'),
(2, 'Mouse',      'Electronics', 500.00,   20,  4.2, '2024-01-02', 'Sol
d'),
(3, 'Chair',      'Furniture',   3000.00,  8,   4.5, '2024-01-02', 'Pend
ing'),
(4, 'Desk',       'Furniture',   7200.00,  6,   4.1, '2024-01-03', 'Sol
d'),
(5, 'Bottle',     'Accessories', 150.00,   40,  3.9, '2024-01-03', 'Retu
rned'),
(6, 'Cox''s Pen', 'Accessories', 20.00,    100, 4.6, '2024-01-03', 'Retu
rned');
```

Tips: Escape apostrophe by doubling: `'Cox''s Pen'` • Dates format: `'2024-01-01'`

Q3 PART 2: solution.php FULL REUSABLE ENGINE

SOLUTION.PHP

```
<?php
$conn = new mysqli("localhost", "root", "", "exam_db");
if ($conn->connect_error) die("Connection failed: " . $conn->connect_err
or);

// 1. Dynamic SELECT Table Printer
function showTable($conn, $title, $sql) {
  echo "<h4>$title</h4>";
  $res = $conn->query($sql);
  if (!$res) { echo "Error: " . $conn->error . "<br>"; return; }
  if ($res->num_rows > 0) {
    echo "<table border='1' cellpadding='4'><tr>";
    while ($f = $res->fetch_field()) echo "<th>{$f->name}</th>";
    echo "</tr>";
    while ($row = $res->fetch_assoc()) {
      echo "<tr>";
      foreach ($row as $v) echo "<td>$v</td>";
      echo "</tr>";
    }
    echo "</table>";
  } else echo "0 results found.<br>";
}

// 2. Reusable UPDATE Runner
function runUpdate($conn, $title, $sql) {
  echo "<h4>$title</h4>";
  if ($conn->query($sql) === TRUE) {
    echo "Updated successfully. Rows: " . $conn->affected_rows . "<br>";
  } else echo "Error: " . $conn->error . "<br>";
}

// 3. Single Value Printer (Total Net Worth, Count)
function showSingle($conn, $title, $sql, $col) {
  $res = $conn->query($sql);
  if ($res && $row = $res->fetch_assoc()) {
    echo "<p><b>$title: </b>" . number_format($row[$col], 2) . " BDT</p
>";
  }
} ?>
```

ZERO-CRASH EMERGENCY RULES & GOTCHAS

- **WHERE vs HAVING:** `WHERE` filters rows; `HAVING` filters groups. Never put `COUNT(*)` or `AVG()` in `WHERE`!
- **SQL Order:** `WHERE > GROUP BY > HAVING > ORDER BY`.
- **Execution Order:** Run tasks in exact sequence. Early updates affect later queries.
- **SQL Not Equal:** Use `<>` or `≠`. String in single quotes `'Text'`.
- **XAMPP Run:** Always run via `http://localhost/folder/index.html` (never double click file!).

EXAM WORDING → EXACT SQL MAPPING DICTIONARY

When Question Asks:	Exact SQL Command to Write:
"For each category / department / spot"	`GROUP BY Category` (must be in SELECT too)
"Total number of ... in each ..."	`SELECT Category, COUNT(*) AS Total FROM items GROUP BY Category`
"Only groups with more than N / avg > X"	`HAVING COUNT(*) > N` or `HAVING AVG(Rating) > 4.0`
"Sorted highest to lowest / descending"	`ORDER BY col_name DESC`
"Spot / Item contains word 'Pen' or 'Beach'"	`WHERE ItemName LIKE '%Pen%'`
"Rating between 4.0 and 4.5 (inclusive)"	`WHERE Rating BETWEEN 4.0 AND 4.5`
"If price < 1000 and status != 'Returned' set 'Low'"	`UPDATE items SET Status='Low' WHERE Price < 1000 AND Status <> 'Returned'`
"Add 5000 bonus, only if resulting ≤ 60,000"	`UPDATE items SET Price = Price + 5000 WHERE Price > 3000 AND (Price + 5000) ≤ 60000`
"10% bonus revenue / Increase by 10%"	`UPDATE items SET Price = Price * 1.10 WHERE Condition`
"Apply 10% discount / Reduce by 10%"	`UPDATE items SET Price = Price * 0.90 WHERE Condition`
"Total net worth of entire store"	`SELECT SUM(Price * Stock) AS NetWorth FROM items`
"Label 'Top Seller' if above own category avg"	`CASE WHEN Price > (SELECT AVG(Price) FROM items t2 WHERE t2.Category = items.Category) THEN 'Top Seller' ELSE 'Regular' END`
"Change two columns at once"	`UPDATE items SET Rating = Rating + 0.2, Price = Price * 1.10 WHERE Condition`

ALL 15 SQL QUERY ARCHETYPES (TAGGED WITH PAST PAPERS 242-261)

15 ARCHETYPES

#	Type & Papers	Exact SQL Query (Runs on items table)
1	Count rows per group 243 251 253 261	`SELECT Category, COUNT(*) AS Total FROM items GROUP BY Category;`
2	Total & avg per group 242 252	`SELECT Category, SUM(Price) AS Revenue, AVG(Price) AS AvgPrice FROM items GROUP BY Category;`
3	Groups with > 1 row 253	`SELECT Status, COUNT(*) AS Total FROM items GROUP BY Status HAVING COUNT(*) > 1;`
4	Top 2 days by date 242	`SELECT SaleDate, SUM(Price) AS Revenue FROM items WHERE SaleDate BETWEEN '2024-01-01' AND '2024-01-31' GROUP BY SaleDate ORDER BY Revenue DESC LIMIT 2;`
5	OR with LIKE substring 261	`SELECT ItemName, Category, Rating FROM items WHERE Rating > 4.5 OR ItemName LIKE '%Pen%' ORDER BY Rating DESC;`
6	Multi-condition filter 261	`SELECT ItemName, Price FROM items WHERE Price > 400 AND Category = 'Electronics';`
7	Whole-table Net Worth 261	`SELECT SUM(Price * Stock) AS NetWorth FROM items;`
8	Count + Avg + Total; HAVING 261	`SELECT Category, COUNT(*) AS Items, AVG(Rating) AS AvgRating, SUM(Stock) AS TotalStock FROM items GROUP BY Category HAVING AVG(Rating) > 4.4 ORDER BY AvgRating DESC;`
9	HAVING calculation 261	`SELECT Category, SUM(Price * Stock) AS Value FROM items GROUP BY Category HAVING SUM(Price * Stock) > 100000 ORDER BY Value DESC;`
10	Distinct dates in HAVING 242	`SELECT Category, AVG(Stock) FROM items GROUP BY Category HAVING COUNT(DISTINCT SaleDate) ≥ 2;`
11	Relative avg (Top Seller) 252	`SELECT i.ItemName, i.Price, CASE WHEN i.Price > (SELECT AVG(i2.Price) FROM items i2 WHERE i2.Category = i.Category) THEN 'Top Seller' ELSE 'Regular Seller' END AS Status FROM items i;`
12	Conditional UPDATE <> 243 251 252 253	`UPDATE items SET Status = 'Low' WHERE Price < 1000 AND Status <> 'Returned';`
13	Bonus with ceiling cap 243 251	`UPDATE items SET Price = Price + 5000 WHERE Price > 3000 AND (Price + 5000) ≤ 60000;`
14	10% discount / raise 252 253 261	`UPDATE items SET Price = Price * 0.90 WHERE Category = 'Furniture'; -- raise: * 1.10`
15	Update 2 columns at once 261	`UPDATE items SET Rating = Rating + 0.2, Price = Price * 1.10 WHERE Rating BETWEEN 4.0 AND 4.5 AND Price > 0;`